

L7 总结

L7 总结:RAG 与向量数据库

JD 权重 8/10,阶段二最高频,面试必问。也是学习者绕了整门课的问题 (L1 “模糊处理”、L3 “按需检索工具”、L5 “取哪几条最相关”、L6 “外置”策略——全指向 RAG)。

一、核心概念

1. 为什么需要 RAG:LLM 三个先天缺陷

知识冻结(训练截止)、不知道私有数据、塞不进上下文(L6 的账)。> RAG = 回答前先去外部知识库“查资料”,把相关内容塞进上下文再作答(开卷考试)。

2. 两个阶段(全课最重要的图)

- 离线建索引(平时一次):文档 → 切块 → embedding → 存向量库
- 在线检索生成(每次提问):问题 embedding → 向量库找 top-k → 塞 prompt → grounding 生成

3. embedding = 文本变“语义坐标”

语义相近 → 向量相近。“马铃薯 vs 土豆”字面零重叠、语义相同,关键词匹配必漏,embedding 能命中。text-embedding-3-small 是 1536 维。

4. 检索 = 余弦相似度 + top-k;向量库负责“找得快”

- 三个部件三种职责:向量库负责“找得快”(HNSW 把 $O(N)$ 降到 $\sim O(\log N)$)、top-k 负责“塞得少”、LLM 负责“答得好”。别混。
- 选型:Chroma(嵌入式/零配置/中小规模)、Milvus/Qdrant(分布式/大规模)、pgvector(数据在 PG)、FAISS(极致速度/纯库)。

5. chunking = RAG 成败关键(本节亲手验证)

- 固定窗口 size+overlap:overlap 防“意思被切在边界丢失”;但硬切字符会切碎句子/跨段落。
- 按语义边界切(标题/段落)远好于硬切。生产用 RecursiveCharacterTextSplitter / MarkdownHeaderTextSplitter。

6. grounding = 检索到 ≠ 模型会用(回扣 L6)

必须强约束:“只依据资料回答,没有就说暂无,不得编造”。否则模型无视检索、凭记忆瞎编(RAG 最致命幻觉)。

二、作业:用真向量库 ChromaDB 搭最小 RAG

homework/rag.py ,4 个环节亲手串起来(可写进简历、面试能讲): - **chunk_text**(固定窗口+overlap)/**chunk_by_heading**(按标题切) - **embed_texts**:OpenAI text-embedding-3-small,批量调用 - **build_index / retrieve**:ChromaDB 存与查(自带 embedding,向量库只管存查) - **rag_answer**:grounding 强约束生成(DeepSeek) 技术栈:embedding=OpenAI + 生成=DeepSeek(解耦,两个 client)+ 向量库=Chroma。

跑通验证: - “咋退款啊要几天”(口语)→ 命中退款条款,精准作答(embedding 语义检索) - “退货运费谁出” → 命中运费规则 - “你们卖不卖苹果手机”(库里没有)→ “暂无相关信息”(grounding 挡住幻觉)

三、亲手验证:chunking 决定 RAG 上限(本节最值钱的体感)

同一份 embedding + 向量库 + prompt,只把切法从固定窗口换成按标题切: || 固定窗口(3 块) | 按标题切(5 块) | |—|—|—| | 块质量 | 切碎句子、跨段落(“售。” 残块) | 每块=一个干净政策小节 || 检索 | k=3=总数,每次返回全部,看不出筛选 | 5 块>k=3,只取最相关 3 块并排序(退款问题→退款块排第1) || 答案 | “怎么退货” 飘忽 | 利落准确,且如实指出库里缺的部分 | > 结论:embedding/向量库/grounding 全没动,只换 chunking,检索与答案质量都上台阶。 > “RAG 效果差先查 chunking” —— 面试讲 RAG 优化的硬通货。

四、生产 RAG 还有一整层(埋点,面试能讲“还能怎么优化”)

rerank 重排、hybrid 检索(向量+BM25)、query rewrite、metadata 过滤、RAG 评估(recall@k/忠实度→L12)、embedding 缓存+重试(→L13)、来源引用。 + 大知识库:embedding 要分批(单次约 2048 上限);id 用内容哈希做增量更新/去重。

五、工程环境:本节顺带迁到 uv(conda → uv)

- `uv init` + `uv add openai chromadb` + `uv add --dev pytest ; .venv` 在仓库内。
- **pyproject.toml + uv.lock + .python-version 入库**,双机 `uv sync` 一键复现(解决之前 conda 手动同步)。
- 新 workflow: `uv run python xxx.py` (自动用 `.venv`)/ `uv add x` (替代 `pip`)/ `uv sync` (按 `lock` 复现)。
- 踩坑:VSCode 要开在仓库根目录(否则推荐错的 `.venv`);chromadb 首次导入慢别 `Ctrl+C`; Windows 控制台中文乱码加 `PYTHONUTF8=1 ;rm -rf` 是 `bash`、PowerShell 用 `Remove-Item -Recurse -Force`。

六、Quiz(3 题精简版,概念主线已在课堂问答验证):chunking/grounding/检索机制,均达标。

七、学习者亮点与短板

- **亮点**:自己抓出“toy 向量库 vs 生产工具”的矛盾(判断力),主动迁 uv,问出“`embed_texts` 为何不引用 `chunk_text`”(职责分离直觉)。
- **短板第 5 次**:`build_index` 漏写 `return collection` (函数干完活忘交出结果)——已再次强调自查仪式。

八、埋点

- **L8 ReAct · L9 LangChain**(chunking 换成成熟 splitter、RAG pipeline 用框架重写)
- **阶段二 capstone**:RAG 做成工具接进客服 agent(模型编排)+ 单元测试系统学习
- **L12 评估**(RAG 指标)· **L13**(embedding 重试/缓存、API 层限流)